

Module 21: Asynchronous Apex & Dynamic Apex

Course Objectives

At the end of this course, you will be able to:

- Understand Asynchronous Apex
- Implement Future Methods
- Understand Queueable Apex
- Implement Batch Apex
- Write a Batch Class
- Scheduling the Batch Class
- Understand Dynamic Apex

21.1 Asynchronous Apex

21.1.1 Future Methods

21.1.2 Queueable Apex

21.1.3 Batch Apex

21.1.4 Schedulable Apex

21.2 Dynamic Apex

Asynchronous Apex

What is Asynchronous Apex?

Asynchronous Apex is a programming technique that allows developers to execute processes in a separate thread at a later time.

The purpose of Asynchronous Apex is to enable developers to

- 1. Execute long-running operations,
- 2. Processing larger data sets,
- 3. Processing resource-intensive tasks,

in the background, hence improving the performance and responsiveness of Salesforce applications.

Asynchronous Apex

Differences between Synchronous Apex and Asynchronous Apex

Synchronous Apex	Asynchronous Apex
1. It is executed immediately	1. It is executed in background
2. User waits for the execution to complete	2. User does not wait as it supports parallel execution
3. It executes in single transaction	3. It is executed in separate transactions
4. We can execute 100 SOQL queries per transaction	4. We can execute 200 SOQL queries per transaction
5. Synchronous Apex can use up to 6 MB of Heap size	5. Asynchronous Apex can use up to 12 MB of Heap size
6. It is used for processing smaller data sets and less complex processes	6. It is used for processing larger data sets and more complex processes which are resource-intensive

Methods of implementing Asynchronous Apex

1. **Future Methods:** These methods are executed asynchronously in the background.
2. **Queueable Apex:** This is like future methods but with enhanced capabilities such as job chaining.
3. **Batch Apex:** This is used for processing large datasets by breaking them into smaller chunks and processing them in batches
4. **Schedulable Apex:** This allows the scheduling of Apex classes to run at specific times, useful for recurring tasks like data maintenance and reporting.

Use Cases of Asynchronous Apex

Below are the Use Cases of Asynchronous Apex which can be performed in the background:

1. Processing large sets of data that cannot be handled within the limits of synchronous Apex.
2. Performing callouts to external services where a response is not immediately required.
3. Chaining together complex functionalities.
4. Scheduling the recurring processes like Generating Reports, Data Clean up, archiving old records, data synchronization, data transformation.
5. Performing complex calculations: Complex calculations takes time; hence it is suggested to process asynchronously.

Future Methods

What is a Future Method ?

Future method is used to perform operations **later** in a separate thread or when the resource is available.

Characteristics of Future Methods:

1. It is asynchronous
2. It must be defined as 'static'
3. It must be annotated with '@future' annotation
4. It cannot return values. The return type of future method is always 'void'
5. Future methods can only take Primitive data types as parameters like integer, double, long, string, Boolean, date, datetime, id and lists of these types.

Sample Code of Future Method

Step 1: To Create a future method, We use the annotation '@Future'.

```
public class FutureMethodExample {  
  
    @future  
    public static void updateCandidateCity(Set<Id> CandidateIds) {  
        // Processing below code asynchronously  
        List<Candidate__c> CandidatesToUpdate = [SELECT Id, Name, city__c FROM Candidate__c WHERE Id IN :CandidateIds];  
  
        for(Candidate__c can : CandidatesToUpdate) {  
            can.city__c = can.city__c + ' - Updated';  
        }  
  
        update CandidatesToUpdate;  
    }  
}
```

Sample Code of Future method

Step 2: Calling Future Method from another class

```
public class CandidateClassFuture {  
  
    public void invokeFutureMethod() {  
        Set<Id> canIds = new Set<Id>();  
        canIds.add('a01IT00002cjtFWYAQ');  
        canIds.add('a01IT00002cjtJwYAI');  
  
        // Call the future method  
        FutureMethodExample.updateCandidateCity(canIds);  
    }  
}
```

Step 3: Executing the method in Anonymous Window

Enter Apex Code

```
1 CandidateClassFuture ob=new CandidateClassFuture();  
2 ob.invokeFutureMethod();
```

Sample Code of Future method

Step 4: Track the status of execution of Future methods from setup→ Apex Jobs

 **Apex Jobs**

[Click here to go to the new batch jobs page](#)

Apex Jobs

Monitor the status of all Apex jobs, and optionally, abort jobs that are in progress.

 **Percent of Asynchronous Apex Used: 0%**
You have currently used 2 asynchronous Apex operations out of an allowed 24-hour organization limit of 250,000. To learn about how this limit is calculated and what contributes to it, see the [Lightning Platform](#).

View: All [Create New View](#)

Action	Submitted Date	Job Type	Status	Status Detail	Total Batches	Batches Processed	Failures	Submitted By	Completion Date	Apex Class	Apex Method
	19/06/2024, 10:20 pm	Future	Completed		0	0	0	Chawla, Sakshi	19/06/2024, 10:20 pm	FutureMethodExample	updateCandi
	19/06/2024, 10:18 pm	Future	Failed	First error: Update failed. First exception on row 0 with id a011T00002cjJwYAI: first error: FIELD_CUSTOM_VALIDATION_EXCEPTION: State/Province should have only 2 characters. [State_Province__c]	0	0	0	Chawla, Sakshi	19/06/2024, 10:18 pm	FutureMethodExample	updateCandi
	19/06/2024, 10:16 pm	Future	Failed	First error: SObject row was retrieved via SOQL without querying the requested field: Candidate__c.City__c	0	0	0	Chawla, Sakshi	19/06/2024, 10:16 pm	FutureMethodExample	updateCandi

Queueable Apex

What is Queueable Apex ?

Queueable Apex is a way to run asynchronous processes just like future methods, providing more flexibility and efficiency as compared to future methods. It allows for job chaining and better handling of complex operations.

Characteristics of Queueable Apex:

1. **Interface Implementation:** Implements the 'Queueable' interface.
2. **Job Chaining:** We can chain one job to another to run complex operations in sequence.
3. **Governor Limit:** More governor limits than future methods, such as higher heap size and CPU time.
4. **Complex data types:** Unlike Future methods, queueable apex can handle complex data types as we can pass **objects** to a queueable class.

Syntax for Queueable Apex

Step 1: Create a class which contains the first job and has the method 'System.enqueueJob()', which calls another job.

```
public class QueueableApex {  
    public static void executeJob() {  
        // Code  
        // Chain another job  
        System.enqueueJob(new SecondTask());  
    }  
}
```

Syntax for Queueable Apex

Step 2: Create a class which contains the second job.

```
public class SecondTask implements Queueable {  
    public void execute(QueueableContext context) {  
        // another code  
    }  
}
```

Syntax for executing Queueable Apex

Step 3: Call the method of the class which contains the first job.

Enter Apex Code

```
1 QueueableApex.executeJob();
```

Step 4: Check the status of execution of Queueable Apex class from Setup→ Apex Jobs

SETUP

Apex Jobs

Click here to go to the new batch jobs page

Apex Jobs

Monitor the status of all Apex jobs, and optionally, abort jobs that are in progress.

Percent of Asynchronous Apex Used: 0%

You have currently used 0 asynchronous Apex operations out of an allowed 24-hour organization limit of 250,000. To learn about how this limit is calculated and what contributes to it, see the [Lightning Platform](#)

View: All Create New View

Action	Submitted Date	Job Type	Status	Status Detail	Total Batches	Batches Processed	Failures	Submitted By	Completion Date	Apex Class	Apex Method
	21/06/2024, 12:07 am	Queueable	Completed		0	0	0	Chawla, Sakshi	21/06/2024, 12:07 am	SecondTask	

Demo

21.1.2 Queueable Apex



Write an Apex Class which will implement 'Queueable' interface to update the position owner of all closed positions to 'Clark Kentman' asynchronously.

Batch Apex

What is Batch Apex ?

Batch Apex is a feature that allows developers to handle large volumes of data by breaking down the processing into smaller chunks, or "batches." This is useful for operations that need to process thousands or millions of records, which would otherwise exceed the governor limit of standard synchronous Apex code.

Batch Class Interfaces

Batch Apex class use specific interfaces to manage batch processing as below:

1. **Database.Batchable interface:** This is the primary interface which defines the methods required for batch processing. It requires three methods:

- **Start():** Used to define the scope of records to be processed.
- **Execute():** Used to process each batch of records.
- **Finish():** Used to process the tasks after all batches have been executed.

2. **Database.Stateful interface:** By default, batch apex is stateless which means each batch is executed independently. By using this interface, we can maintain the state across batches in a batch apex job.

3. **Database.AllowsCallouts interface:** This interface is used when the batch job needs to make external web service callouts.

Asynchronous Apex: Parallel Execution.

WE can implement it in 4 ways:

1. Future Methods
2. Queueable Apex
3. Batch Apex

100 records: I will create 5 batches of 20 records each.

Batch 1 - 20 records

Batch 2 - 20 records

Batch 3 - 20 records

Batch 4 - 20 records

Batch 5 - 20 records

Syntax of Batch Class

Database.Batchable interface is implemented here.

```
global class BatchClass1 implements Database.Batchable<subject>{  
  
    global Database.QueryLocator start(Database.BatchableContext bc) {  
        | return Database.getQueryLocator(Query);  
    }  
  
    global void execute(Database.BatchableContext bc, List<subject> scope) {  
        //code  
    }  
  
    global void finish(Database.BatchableContext bc) {  
        | System.debug('Batch job completed successfully.');    }  
}
```

Note: Database.BatchableContext interface is used to track the batch jobs and get the relevant information about the jobs like jobId etc.

Syntax of calling Batch Class

Call the Batch class using 'ExecuteBatch()' method, with the parameters as object of batch class and the batch size.

```
BatchClass1 ob= new BatchClass1();  
database.executeBatch(ob,5);|
```

Minimum Batch Size → 1 record

Maximum Batch Size → 2000 records

Default Batch Size → 200 records

Schedulable Apex

Scheduling a Batch class using 'Schedulable' interface

We can schedule the Batch Apex classes or any other Apex Class to run at specific times using the 'Schedulable' interface. This is useful for tasks that need to be performed at regular intervals.

Key aspects of Schedulable interface:

1. To schedule the Apex class, the class must implement 'Schedulable' interface.
2. This interface needs to implement *execute* method, which contains the logic that we want to schedule.

Scheduling a Batch class using 'Schedulable' interface

To Schedule an Apex Class manually, follow below steps:

1. Create a class which will implement 'Schedulable' interface.
2. Define the *execute* method in this class. Call the method of the class (which we want to schedule) inside the execute method.
3. Go to setup → Apex Classes → Schedule Apex and search for the class (we want to schedule) which has implemented Schedulable interface.
4. Select the Schedule and Save.

Scheduling a Batch class using 'Schedulable' interface

If we want to schedule the batch class we created earlier, we need to create a class which will implement 'Schedulable' interface and call the Batch class inside the execute method.

```
global class SchedulePosition implements Schedulable{  
    global void execute(SchedulableContext sc) {  
        BatchApexPosition batch = new BatchApexPosition();  
        Database.executeBatch(batch);  
    }  
}
```

Calling the Batch Class to Schedule

Schedulable Apex

Using CRON Expression

A cron expression is used to define the schedule for the execution of the job.

String cronExp = '0 30 10 ? * MON'; // Every Monday at 10:30am

String jobId = System.schedule('Weekly Job', cronExp, new SchedulePosition());

A cron expression has the following format:

Seconds Minutes Hours Day_of_month Month Day_of_week Year

Seconds(0-59)

Minutes(0-59)

Hours(0-23)

Day_Of_Month(1-31)

Month(1-12 or JAN-DEC)

Day_of_Week(1-7 or SUN-SAT)

Year(Optional)

Dynamic Apex

Accessing sObject in Dynamic Apex

We can access the sObject in Dynamic Apex using the '**Schema**' class. This is used when we need to write the code that can handle different types of sObjects at runtime.

Functions used in the **Schema** class:

1. **Schema.getGlobalDescribe()** → It is used to retrieve metadata information about all SObjects (standard and custom objects) defined in our Salesforce organization.
2. **Schema.getDescribe()** → It is used to obtain metadata information about any object, field or any other schema component. This method is available on various schema classes like schema.sObjectType, schema.sObjectField.

What is Dynamic SOQL?

Dynamic SOQL is used to create and execute the SOQL queries at run time using strings rather than hard coding them into the Apex code.

```
String objectName = 'position__c';
```

```
String fieldName = 'Name';
```

```
String searchTerm = 'Developer';
```

```
String query = 'SELECT Id, ' + fieldName + ' FROM ' + objectName + ' WHERE ' + fieldName + ' LIKE \'' + searchTerm + '%\"';
```

```
List<SObject> records = Database.query(query);
```